



Conversational Recommender System for Vehicles

Primož Gabrovec, Rožle Sterle and Zala Urh

Abstract

In this report, we design a Conversational Recommender System for vehicle selection. The system helps users express their preferences in natural language and translates them structured technical terms. We propose a dual data strategy that combines two data retrieval approaches. The first is constraint-based filtering over a structured SQL-like database of vehicle attributes. The second is FAISS-based similarity search over vector embeddings generated from vehicle brochures. We evaluate the proposed system against multiple variants. LLM such as Qwen and Llama without any data retrieval represents the baseline. More advanced comparison represent a non-conversational version of the proposed recommender system. In all cases the proposed recommender system outperformed the others.

Keywords

Conversational Recommender System, RAG, FAISS, Car recommendation, LLM

Advisors: Aleš Žagar

Introduction

In the past years, choosing a suitable vehicle has become very challenging due to a wide range of options. Customers can choose between many models with different fuel types, engine characteristics, size, and other specifics.

Most existing systems that assist users in selecting vehicles are based on predefined filters, such as price range, brand, transmission types, and others. While this approach is effective for experienced users, it might not be that suitable for those who cannot easily translate their needs into technical details.

As an improvement to such systems, Conversational Recommender Systems (CRS) have been introduced. A CRS is an interactive system that allows users to express their preferences through dialogue, while the system can ask follow-up questions to refine these preferences and provide the best recommendation(s).

In this paper, we propose a hybrid CRS for vehicle selection using that combines large language models (LLMs), structured constraint extraction, and retrieval-augmented generation (RAG). The system engages the user in a conversation with an LLM that helps them express and refine their vehicle preferences. The final product is a system that simplifies making a decision for non-expert users, and also improves customer experience in vehicle selling companies.

Related work

Several studies have explored recommender systems, however the vehicle domain remains limited. We categorized existing work into recommendation systems for vehicles and conversational recommender systems for various products.

In the context of vehicle recommendation, Beddoua et al. [1] proposed a system using BERT4Rec, which predicts which cars a user might be interested in based on past interactions (without any user communication). Singh et al. [2] introduced a one-turn dialogue system that uses LDA model to analyse user reviews, where users describe their car preferences and the system returns top few recommendations.

For general CRS, several multi-turn dialogue systems have been proposed, especially in the travel domain. Liao et al. [3] proposed a deep recommender system using seq2seq with a neural latent topic component. Baizal et al. [4] introduced an ontology-based system, and Carrillo et al. [5] introduced a system that uses Chatbot to collect user preferences and generates similarity-based recommendations using SBERT embeddings.

In addition to the travel domain, CRS have been implemented in several other domains. Sun et al. [6] developed a reinforcement learning based CRS for restaurant recommendation using Yelp review data. Zhu et al. [7] proposed a movie recommendation CRS using retrieval-augmented generation (RAG) approach in combination with a large language model.

Methods

The proposed system is designed as a Conversational Recommender System (CRS) that integrates large language models (LLMs), structured data processing, and retrieval-augmented generation (RAG) to provide personalized vehicle recommendations through natural language interaction. The overall pipeline is described below as well as shown on the figure 1.

Data Preprocessing

The first dataset was extracted from the CarAPI website [8]. It provides vehicle data in JSON format, such as price, fuel consumption, engine specifications... Data was then processed and stored into structured SQLite database (on Figure 1 marked as Structured database).

The second dataset consists of vehicle brochures in PDF format extracted from the Auto-Brochures website [9]. The dataset includes multiple car brands available on the US market, of which 26 were selected for this work. We used a Qwen embedding model to generate vector representations of the dataset content. These embeddings were then indexed using FAISS (Facebook AI similarity search) [10] to enable efficient similarity search (Dataset is marked on Figure 1 as FAISS Vector Database).

Conversational Recommender System - CRS

The system begins with user describing in natural language their preferences and requirements they look for in a new car. That input is then parsed by the LLM (on figure 1 marked as LLM Parser).

Parsing is implemented by first extracting relevant attributes from the user's query using the embeddings of the description of the attributes. When we defined a schema for the structured database, we also crafted metadata for each attribute in a form of a short description. This description also includes synonyms, related terms, user intentions and example queries. Then we embed those descriptions using an embedding model, which was selected with benchmarking of different models, with a specialised instructions for the task. When the user query is received, we embed it using the same embedding model and then calculate similarity between the query embedding and the attribute description embeddings. The attributes with the highest similarity are selected as relevant to the user's query.

For each relevant attribute, the LLM is then prompted to extract the corresponding value and constraint, which proved to be more effective than directly prompting the LLM to extract every attribute at once.

Table 1 shows an example of the extracted structured information from a user query. The LLM is able to identify that the user is looking for an SUV with electric engine, but it cannot determine the exact value of the fuel consumption constraint, which is then marked as None.

Name	Value	Constraint (min, max, equal...)
body_type	SUV	equal
engine_type	electric	equal
combined_l_per_100km	None	None

Table 1. Example of parsed text for the following user's query: "I'm looking for a spacious SUV with low consumption, preferably electric."

That is why a single user query is usually not enough to generate a good recommendation. User also might not express all their preferences in a single query, leading to a very large choice of cars.

To address these issues, we improved the system with creating a conversational LLM (marked on Figure 1 with the same name). This model interacts with the user to complete missing constraints and suggest additional attributes that could be in interest to the user.

Then the system employs a dual data strategy using databases described in the previous section. First, we fetch relevant cars from the database using constraint-based filtering, which yields the first candidate set. Next we embed the user's query, along with the system's conversation history, and use it for semantic search in the FAISS vector database. Which contains embedded chunks from the vehicle brochures. This allows the system to retrieve cars that are semantically similar to the user's preferences, even if they do not exactly match the structured constraints. While we can extract information such as number of seats, or fuel type from the structured database, other information such as a car's feel, design or equipment is not easily captured in structured form, but can be retrieved from the brochures using the semantic search.

The system then combines both retrieved sets of car candidates and filters them again to ensure that candidates from the FAISS search also meet the user's constraints. The context of the retrieved chunks is then correctly formatted and provided to the LLM as part of the final recommendation prompt. So the LLM can summarize the retrieved information and generate a final recommendation response. The LLM also comments on the key differences between the suggested cars, which can help users make an informed decision.

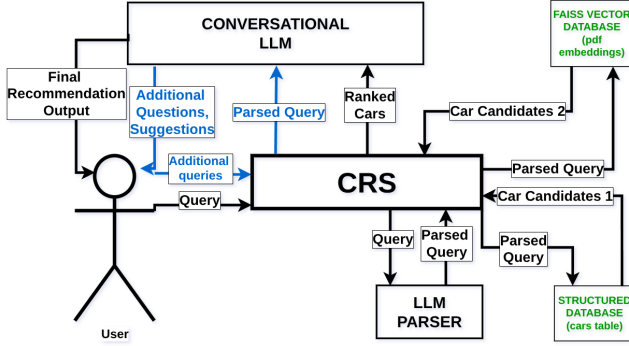


Figure 1. CRS Architecture. Blue arrows represent conversation interaction loop between the user and the LLM, green components represent databases that the system uses.

Evaluation methods and Results

We designed two types of tests: the ones that evaluate the embedded model, and the ones that evaluate the final system.

Attribute embeddings evaluation

We designed a benchmark to evaluate the performance of different embedding models for the task of identifying relevant attributes from user queries. We created a dataset of 40 user queries, each manually annotated with the relevant attributes. We then tested several embedding models, from Qwen, BAAI-bge, nomic-ai, GTE, Google’s gemma and jinaai. To rank the models we measured Spearman’s correlation to measure how well the model’s ranking of attributes agrees with the ground truth (Agreement). We also measured the separation with Cohen’s d , since we want a large distance between the relevant and irrelevant attributes, and finally we also measured the F1 score, which was also optimized with different thresholds.

Results of evaluations of the embedded models with their optimal thresholds are shown in the Table 2. In the end, Qwen3-Embedding-0.6B model was chosen since it was the best trade-off between its performance and speed.

Model	Threshold	Agreement	Separation	F1	Final score
Qwen/Qwen3 Embedding-4B	0.57	0.40	1.65	0.57	0.87
jinaai/jina-embeddings v5-text-small	0.33	0.39	1.61	0.52	0.84
Qwen/Qwen3 Embedding-0.6B	0.49	0.40	1.58	0.53	0.84
google/embeddinggemma 300m	0.29	0.39	1.59	0.51	0.83
jinaai/jina-embeddings v5-text-nano	0.33	0.38	1.55	0.54	0.83
...	...	...	...	...	...

Table 2. Results of performances of different embedding models

Model evaluation

Evaluating language models is challenging because their outputs are not fully deterministic, especially when they are in conversation mode. We designed three different tests to evaluate the system and the models.

First, we evaluate whether the system performs better than a baseline. We manually created a set of user queries and manually parsed them into constraints (similar to Table 1). We then ran our system in non-conversational mode, which directly returns a set of recommended cars. This result is compared to the set of cars obtained from the structured database using the manually verified constraints. From this comparison, we compute standard metrics such as number of correctly retrieved cars, missed cars, precision, recall, and F1 score. We also run the same evaluation for a baseline LLM without any system support.

The second evaluation tests the conversational setup. This evaluation is not fully automated. We use the same queries as before, but divide them into few parts. We simulate multi-turn interaction by providing additional part after each LLM question. After the conversation ends, we store the final set of recommended cars and compare it to the expected set obtained from the structured database using the manually parsed query. We also compare that to two baselines

Results of the two tests:

Model	number of correctly retrieved cars	number of missed cars	recall
baseline LLM Qwen	2	15	0.12
CRS non-conversational Mistral	0	17	0.00
CRS non-conversational Qwen	14	3	0.82
CRS conversational Qwen	14	3	0.82

Table 3. Results of the first and second evaluation tests

The third evaluation tests whether the RAG component is working correctly. After each recommendation, we manually check whether the returned cars satisfy the user’s stated preferences.

Model	number of user’s preferences	number of satisfied user’s preferences	percentage
baseline LLM	5	3	0.6
CRS non-conversational	5	5	1.0
CRS conversational	5	5	1.0

Table 4. Results of the third evaluation tests

Example conversation

We also present one of the example conversation flow between user and our system:

User: I am looking for an affordable family SUV with 7 seats.
Assistant: To help narrow down your options, could you please specify if you prefer a particular make or model? Addi-

tionally, do you have any requirements regarding the fuel type or the seating arrangement like second-row captain's chairs versus a bench seat?

User: I do not have a brand or model preference, but I would like to have a diesel engine.

Assistant: Given your preference for a diesel engine and need for a 7-seater SUV, I recommend focusing on the Audi Q7, Chevrolet Suburban, and Chevrolet Tahoe. These models balance affordability:

1. ****Audi Q7****: Offers robust performance and luxurious interiors suitable for a family. However, note that finding a diesel variant might be limited due to market availability, typically preferring gasoline engines.
2. ****Chevrolet Suburban****: Provides ample space and reliable diesel option, ideal for large families needing extra room. It's known for durability and comfort, making long journeys easier.
3. ****Chevrolet Tahoe****: Another strong contender offering a comfortable ride and sufficient seating capacity. Like the Suburban, it has a diesel version which ensures good fuel efficiency and lower running costs over time.

While the Mercedes-Benz models offer premium features, they tend to be more expensive and less common in diesel versions. The Land Rover Discovery is also a great choice, though it primarily uses petrol engines and is priced higher than the others mentioned here.

For your specific needs, the ****Suburban**** and ****Tahoe**** stand out due to their reliability, spaciousness, and availability of diesel engines, balancing practicality and cost-effectiveness well.

Top Recommendations:

1. Audi Q7
2. Land Rover Discovery
3. Mercedes-Benz GL-Class
4. Mercedes-Benz GLS-Class
5. Chevrolet Suburban
6. Chevrolet Tahoe

Discussion

As seen in the table 3, the proposed recommender system outperforms the baseline LLM across all evaluation metrics. However, the results reflect the advantage of the conversational system, as it achieves the same performance as non-conversational. That is actually expected and as a positive indication. We gave to non-conversational CRS a query, and to conversational CRS part by part of this query. This suggests that the conversation mechanism is functioning as intended

We also evaluated alternative models within the same pipeline. One of them is a smaller model Mistral. This one had a lot

of trouble just by parsing the query, which led to even worse performance compared to the baseline.

From the Table 4 we can once again observe that CRS Systems outperformed baseline LLM. It is worth noting that evaluating the baseline LLM is very challenging, since it may hallucinated some of the car models or cars technical details.

There are many areas in this project that can be improved. A major limitation is the strong dependency on the underlying LLM, particularly for query parsing. Bigger models could improve both robustness and accuracy, while on the other hand smaller models currently fail to operate reliably within the proposed pipeline.

Another limitation of the system is its computational cost and latency, which mainly happens due to the conversational loop and the embedding-based retrieval step.

Overall, the system satisfies the initial requirements by providing a functional conversational recommendation flow with mostly consistent outputs. The results indicate that the proposed hybrid approach improves over a baseline LLM and confirms that conversational recommender systems work well in this domain.

References

- [1] Amira Riham Beddouda and Bochra Belhemissi. *A VEHICLE RECOMMENDATION SYSTEM FOR SALES*. PhD thesis, KASDI MERBAH UNIVERSITY OUARGLA.
- [2] Saumya Singh, Soumyadeepa Das, Ananya Sajwan, Ishanika Singh, and Ashish Alok. Car recommendation system using customer reviews. *International Research Journal of Engineering and Technology (IRJET)*, 9(10):983–989, 2022.
- [3] Lizi Liao, Ryuichi Takanobu, Yunshan Ma, Xun Yang, Minlie Huang, and Tat-Seng Chua. Deep conversational recommender in travel, 2019.
- [4] ZK Abdurahman Baizal, Dede Tarwidi, B Wijaya, et al. Tourism destination recommendation using ontology-based conversational recommender system. *International Journal of Computing and Digital Systems*, 10, 2021.
- [5] Juan Carlos Carrillo, Victoria Beltran, Laura Sebastia, and Eva Onaindia. Smarttur+ eco: a conversational recommender system for tourism. 2023.
- [6] Yueming Sun and Yi Zhang. Conversational recommender system. In *The 41st international acm sigir conference on research & development in information retrieval*, pages 235–244, 2018.
- [7] Yaochen Zhu, Chao Wan, Harald Steck, Dawen Liang, Yesu Feng, Nathan Kallus, and Jundong Li. Collaborative retrieval for large language model-based conversational recommender systems. In *Proceedings of the ACM on Web Conference 2025*, pages 3323–3334, 2025.
- [8] Carapi, 2026.

^[9] Auto-brochures.

^[10] Welcome to faiss documentation, 2026.